

Defect B — Embedded Curated Indexer Providers (design)

Date: 2026-06-12 **Status:** design (operator-approved approach: "curated Go providers first") **Operator directive:** "On device api app MUST have Jackett embedded same we have the API itself" + "Client app has only 4 providers in onboarding wizard! We do support now many many more via Jackett!"
Relationship to Defect A: Defect A (fixed in 0deb54e7) made the client correctly fetch *whatever the api-app serves*. This (Defect B) makes the api-app *serve more than the ~7 natives* — without an external Jackett sidecar — so the on-device experience shows many providers.

Classification: project-specific.

1. Problem

The on-device api-app (`digital.vasic.lava.api`) embeds `lava-api-go` as a c-shared `liblavaapi.so`. Its `GET /providers` catalogue is built from `provider.ProviderRegistry.All()`, which on the embedded path (`internal/mobile/mobile.go newProductionScraperDeps`) registers only the compiled-in native providers (`rutracker`, `nnmclub`, `kinozal`, `archiveorg`, `gutenberg`). The "many more via Jackett" providers require an EXTERNAL Jackett sidecar (`internal/jackett/client.go` → `LAVA_API_JACKETT_URL`), which cannot run inside the on-device embed: Jackett is .NET and cannot be cgo-embedded "same as the API itself".

2. Goal / non-goals

Goal: the on-device api-app's `/providers` exposes a growing, curated set of popular PUBLIC trackers as first-class providers (`SEARCH + MAGNET_LINK`), each a compiled-in Go `provider.Provider`, with zero external dependency and zero config, covered by falsifiable real-stack tests.

Non-goals (explicitly, anti-bluff honesty):

- NOT a generic Cardigann YAML engine (operator-deferred; that is the "from scratch engine" option not chosen).
- NOT "all 500 Jackett indexers". A curated subset. The §6.J-honest framing: every provider that appears in `/providers` MUST actually work end-to-end; we add them as they are built + tested, never as empty declarations (§6.E).
- The external Jackett sidecar path (`internal/jackett/`) stays UNCHANGED for server deployments that want the full Jackett catalogue.

3. Architecture

Each curated provider is an independent Go package under `internal/provider/curated/<name>/` implementing `provider.Provider` by embedding `provider.BaseProvider` (for the catalogue-metadata defaults) and overriding only what differs. Registration happens in BOTH startup paths that build the registry, exactly where the natives register today:

- `internal/mobile/mobile.go` → `newProductionScraperDeps` (embedded on-device).
- `cmd/lava-api-go/main.go` (standalone server binary).

No router changes: the existing `/providers` handler + the per-provider `/v1/{id}/{op}` routes already iterate the registry, so a registered curated provider is automatically discoverable + searchable.

```
GET /providers                → registry.All() → [natives...,  
curated...]    (catalogue)  
GET /v1/{curatedId}/search?q=... →  
registry.Get(id).Search(...)    (results)
```

3.1 Provider unit contract

Each provider package exposes `New()` `provider.Provider` (and an internal `Client` with an injectable `*http.Client` + base URL so tests substitute a `httptest.Server`). It implements:

Method	Curated behaviour
<code>ID()</code>	stable lowercase id, e.g. "thepiratebay", "yts", "eztv"
<code>DisplayName()</code>	human label, e.g. "The Pirate Bay"
<code>Kind()</code>	"native" (compiled-in; honest — not a remote jackett indexer)
<code>SupportsAnonymous()</code>	true (no login)
<code>AuthType()</code>	<code>provider.AuthNone</code>
<code>Encoding()</code>	"UTF-8"
<code>Capabilities()</code>	{ <code>CapSearch</code> , <code>CapMagnetLink</code> } (+ <code>CapTorrentDownload</code> only where the tracker serves a real .torrent)
<code>BaseURLs()</code>	the canonical site URL(s) for informational display
<code>Search(ctx, opts, cred)</code>	real HTTP GET to the tracker's JSON/HTML API; parse → [] <code>SearchItem</code> { <code>Title</code> , <code>InfoHash</code> , <code>MagnetLink</code> , <code>SizeBytes</code> , <code>Seeders</code> , <code>Leechers</code> , ...}
<code>GetTorrent / DownloadFile</code>	<code>ErrUnsupported</code> for magnet-only trackers (§6.E: capability NOT declared)
all other extended methods	<code>ErrUnsupported</code>
<code>Login / CheckAuth / FetchCaptcha</code>	anonymous: <code>Login</code> no-op success, <code>CheckAuth</code> true, <code>FetchCaptcha</code> <code>ErrUnsupported</code>

Method	Curated behaviour
HealthCheck(ctx)	lightweight GET to the API root/health

Magnet construction: from the BitTorrent v1 infoHash + magnet:?xt=urn:btih:<hash>&dn=<title>&tr=<trackers...> using the well-known public trackers list (a package constant — protocol data, not a deployment address).

4. Phase plan

Phase 1 — JSON-API, anonymous, no Cloudflare (highest value / lowest risk / real-testable now):

1. thepiratebay — `https://apibay.org/q.php?q=<query>&cat=0` → JSON array [{name, info_hash, seeders, leechers, size, ...}]. Magnet from info_hash.
2. yts — `https://yts.mx/api/v2/list_movies.json?query_term=<q>` → JSON; each movie's torrents[] carry hash → magnet.
3. eztv — `https://eztvx.to/api/get-torrents?limit=...&keywords=<q>` → JSON torrents[] with magnet_url, seeds, size_bytes.

Phase 2 — Cloudflare-gated HTML (reuses the existing FlareSolverr seam): 4. x1337 (1337x) — HTML search + per-row magnet page; via FlareSolverr when the site is Cloudflare-challenged. 5. torrentgalaxy — HTML; FlareSolverr as needed.

Phase 1 ships first as independent, separately-committable providers. Each provider is one commit (impl + tests + registration), so the catalogue grows incrementally and every increment is releasable.

5. Error contract

Search maps upstream failures to the package sentinels (ErrNotFound/ErrForbidden/ErrNoData/ErrUnknown) — never panics. A provider whose upstream is unreachable returns an error from Search; the registry + /providers catalogue still lists it (the descriptor is static), so a transiently down tracker does not remove the provider from onboarding (it just returns no results until reachable), matching native behaviour. §6.AC: every error path calls observability.RecordNonFatal with {provider, operation, error_class}.

6. Testing (Anti-Bluff Pact — per provider, all mandatory)

1. **Fixture parser test** (<name>_test.go): a captured REAL upstream response intestdata/ → drive Client.Search against an httptest.Server serving the fixture → assert mapped SearchItem fields (title, infoHash, magnet contains the hash, sizeBytes, seeders). FALSIFIABILITY: mutate the field-mapping (e.g. read seeders from the wrong JSON key) → the assertion fails with a clear message. Recorded in the Bluff-Audit stamp.

2. **Real-network test** (`//go:build realtrackers`): hit the LIVE API with a stable query (e.g. "ubuntu") → assert ≥ 1 result with a non-empty 40-hex infoHash + a magnet. Gated so default `go test ./...` makes no outbound calls (mirrors `-PrealTrackers`).
3. **Registry/catalogue test** (`internal/provider` or `internal/handlers`): after registering the curated providers, `GET /providers` body contains the new ids, each with `kind:"native"`, `authType:"NONE"`, `capabilities` $\supseteq$ `["SEARCH", "MAGNET_LINK"]`, and `len(providers) > nativeCount`. This is the load-bearing "more than the natives" assertion the operator's bug is about — on user-visible wire state.
4. **Registration parity test**: assert the `EMBEDDED` path (`mobile.go`) and the `STANDALONE` path (`main.go`) register the SAME curated set (a regression guard so the on-device app never silently lacks a provider the server has — the §6.J "module-green-but-app-broken" class).

7. Constitutional notes

- **§6.R**: API base URLs follow the EXISTING native pattern — a canonical-domain constant per provider package (the same shape as `rutracker.NewClient("https://rutracker.org/forum")`). These are the provider's own identity (protocol endpoints), not deployment addresses; consistent with how all 5 natives already encode their base. No ports/IPs/secrets hardcoded.
- **§6.H**: all Phase-1 providers are anonymous — zero credentials. Phase-2 stays anonymous (public trackers); FlareSolverr URL comes from config (`LAVA_API_FLARESOLVERR_URL`).
- **§6.E**: capabilities declared == methods implemented; unsupported → `ErrUnsupported`.
- **§6.Z / §6.AA / §6.Y**: shipping these to the on-device app is a new api-app + embed build → version bump + the §6.Z device-test gate + two-stage distribute apply at distribute time (separate from landing the code).
- **api-source.hash**: every change to `lava-api-go` source drifts the embed hash; `scripts/compute-api-source-hash.sh > core/apiengine/src/main/resources/api-source.hash` runs after each provider lands (the SourceHash contract gate).

8. Honest scope statement (for CHANGELOG / release notes)

"Adds built-in public-tracker providers (The Pirate Bay, YTS, EZTV, ...) to the on-device API — searchable with magnet links, no external Jackett required. This is a curated, growing set, not a full Jackett mirror; server deployments can still point at an external Jackett for the complete indexer catalogue."